Fraud Detection in Online Transactions

BA 5200

Information Systems Management and Data Analytics

Ak Syracuse, Brynn Beaulieu, Olivia Gette, Rachel Szczepanski, & Sabri Laphitz

Table of Contents

Introduction

The purpose of this project is to develop a machine learning-based fraud detection system for online transactions at LOL Bank Pvt Ltd as well as dive into current analytical standings of the company. As digital banking and mobile payment systems continue to expand, financial institutions face an increasing amount of fraudulent activity, including identity theft, unauthorized access, and suspicious transaction patterns (Bhanusri & Reddy, 2025). These activities result in billions of dollars in global financial losses every year. Industry reports show digital fraud grew by over 25% annually, placing banks at risk of direct financial loss, regulatory penalties, and loss of customer trust (Johora et al., 2024).

The core issue stems from traditional rule-based detection methods struggling to adapt to evolving fraud tactics, often resulting in high false positive rates and delayed detection (Hashemi et al., 2023). These limitations reduce operational efficiency and weaken customer confidence.

To address this issue, this project proposes the development of supervised machine learning models trained on historical transaction data consisting of 200,000 records and 24 variables (Maru, 2024). The system will identify complex patterns in transaction behavior to improve fraud detection accuracy while minimizing false alarms (Shaik, 2025).

The rest of this project evaluates our given solution, and gives a deep analytical insight to the current dataset (Copied From Proposal).

Executive Summary

This project dives into the implementation of a machine-learning based fraud detection system for LOL Bank Pvt Ltd to safeguard its expanding digital and mobile payment infrastructure while also looking into the current analytics of the dataset. As financial cybercrime evolves beyond the capabilities of traditional rule-based systems, LOL Bank faces significant risks, including direct financial losses, regulatory non-compliance, and diminished customer trust. The project analyzes the implementation of supervised machine learning models trained on datasets entailing historical transactions to identify complex, non-linear patterns of fraudulent behavior as well as analytical insights to the current state of the data. By transitioning to a data-driven approach, the bank will be able to achieve real-time detection, reduce the incidence of false positives that disrupt legitimate customer experiences, provide scalable defense against identity theft and unauthorized access, and give themselves a baseline of current state analytics to compare to future state analytics. The project includes a comprehensive cost-benefit analysis and structure timeline for deploying a high-accuracy predictive model and historical trends and data that ensures the long-term security and reputation of LOL Bank.

Background

The landscape of the global financial industry has undergone a radical shift toward "Digital-First" banking, a trend accelerated by the rapid adoption of mobile payment platforms. While this shift has provided unprecedented convenience for consumers, it has simultaneously expanded the "attack surface" for cybercriminals. Traditional fraud detection systems at institutions like LOL Bank have historically relied on static, rule-based logic, such as flagging transactions over a certain dollar amount or those originating from foreign IP addresses. However, modern fraud is increasingly sophisticated, involving synthetic identities and automated bot attacks that mimic legitimate user behavior to bypass these simple thresholds. Recent industry data indicates that digital fraud is growing at an annual rate of 25%, making it a systemic risk rather than an outside operational issue. For LOL Bank, the transition from reactive to proactive security is no longer optional. This project is rooted in the necessity of using advanced analytics to stay ahead of "social engineering" and "account takeover" tactics that currently cost the global economy billions of dollars each year (Copied From Proposal).

Motivation

LOL Bank Pvt. Ltd. is a financial institution that processes a high volume of online and mobile transactions. With the expansion of mobile banking platforms and the acceleration of digital payments during the COVID-19 pandemic, there has been a rapid growth of digital banking. Due to this, banks have experienced an increase in fraudulent activities such as unauthorized access, identity theft, and abnormal transaction patterns. As more financial services moved online, criminals online simultaneously developed more sophisticated techniques to exploit vulnerabilities. To address these risks, LOL Bank has provided a dataset of historical transactions, including customer information, transaction metadata, merchant details, device information, and fraud labels, to support the development of a machine learning-based fraud detection system.

Fraud detection in online transactions refers to the identification of unauthorized, deceptive, or malicious activity within a digital financial system. As online transactions and mobile payments have expanded, financial fraud has become increasingly complex. Criminals now use stolen credentials, automated bots, and social engineering techniques to exploit weaknesses that are found in online systems. Traditional rule-based systems are no longer sufficient to counteract these behaviors (Hilal et al., 2022). This shift has led to the adoption of advanced, data-driven techniques such as machine learning, anomaly detection, and real-time predictive analysis.

Fraud primarily occurs due to financial incentives combined with the weaknesses in transaction monitoring systems. Some key causes include identity theft, account takeover, synthetic fraud, and automated bot attacks. These behaviors often occur without immediate visibility, especially in systems that lack advanced detection mechanisms. Fraud does not follow a uniform pattern; it may spike during high-traffic events like holidays or big sales, appear from unusual locations or

devices, or show through atypical purchasing behaviors relative to past account history. Since fraud tactics evolve constantly, detection systems must also adapt with them (Lulka, 2025).

This problem deserves attention because online transaction fraud has major financial and reputational consequences. Billions of dollars are lost annually by financial institutions and consumers. Users affected by fraud may abandon platforms entirely and cause further financial harm to businesses (Hilal et al., 2022). Being able to detect fraud accurately and efficiently is crucial in both an economic sense as well as a social aspect.

Project Details

The technical core of this project involves the application of the Python programming language to perform advanced data science and predictive modeling. Python was selected as the primary tool due to its extensive ecosystem of specialized libraries, such as Pandas for data manipulation and its ability to implement supervised learning algorithms. Python's ability to integrate seamlessly with existing financial database architectures makes it the industry standard for deploying scalable machine learning solutions. The dataset used for this analysis was provided by LOL Bank Pvt Ltd and consists of 200,000 unique transaction records featuring 24 distinct variables. These variables include:

- Customer Demographics: Information used to establish baseline spending profiles.
- Transaction Metadata: Specifics such as timestamps, transaction amounts, and currency types.
- Technical Identifiers: Device IDs, IP addresses, and merchant details to track geographic and hardware-based anomalies.
- Fraud Labels: Historical indicators used to "train" the model to distinguish between legitimate and fraudulent events.

The primary focus of the analysis is to perform feature engineering and model training to identify hidden correlations between these variables. By analyzing historical behavior, the system will learn to detect "abnormal" patterns, such as a sudden high-value purchase on a new device from an unusual location, in milliseconds. The ultimate goal is to produce a model that maximizes the "True Positive" rate (catching actual fraud) while significantly lowering the "False Positive" rate, thereby ensuring that legitimate customers are not inconvenienced by unnecessary transaction blocks (Copied From Proposal).

Data Understanding

Data Description

The dataset is obtained from Kaggle and downloaded as a CSV file. It has 24 columns and around 200,000 rows, meeting the requirement of at least 10,000 observations and 20 variables. The dataset will be imported into Python using the Pandas library. After importing the file, the

first step will be inspecting the structure of the dataset and identifying data types. We will then identify missing values, delete duplicate records, and ensure variables are in the correct datatype.

The dataset includes transaction data from LOL Bank Pvt. Ltd. to support fraud detection analysis. With the increase of online banking transactions, there are more opportunities for fraudulent activities that could negatively impact the bank and cause financial losses. Therefore, identifying patterns with fraud is critical to help prevent it in the future.

There are customer-level and transaction-level variables included in the dataset.

Customer related variables include: Customer_ID (unique identifier), Customer_Name, Gender, Age, State, City, Customer_Contact, and Customer_Email.

Transaction level variables include: Transaction_ID, Transaction_Date, Transaction_Time, Transaction_Amount, Transaction_Type, Transaction_Description, Merchant_ID, Merchant_Category, and Transaction_Currency.

Account-level and additional variables include: Bank_Branch, Account_Type (savings or checking), Account_Balance, Transaction_Device, Device_Type, Transaction_Location (latitude and longitude).

Lastly, the Is_Fraud column is a binary indicator showing whether a transaction is fraudulent (1) or not (0). This variable will be used as the target variable in the analysis.

Dataset

As the dataset is relatively large with a great number of columns, the link to the dataset can be found here rather than pictures of the entire dataset listed in this report.

Column Definitions

- Customer_ID
- Customer_Name
- Gender
- Age
- State
- City
- Bank_Branch
- Account_Type
- Transaction_ID
- Transaction_Date
- Transaction_Time
- Transaction_Amount
- Merchant_ID
- Transaction_Type

- Merchant_Category
- Account_Balance
- Transaction_Device
- Transaction_Location
- Device_Type
- Is_Fraud
- Transaction_Currency
- Customer_Contact
- Transaction_Description
- Customer_Email

Data Cleaning & Exploration

The dataset used for this project didn't need to be cleaned much before using. The only cleaning that needed to be done was changing the data type for the Transaction_Date and Transaction_Time columns to datetime instead of string (which is what they were stored as originally). All the analyzing for this project was done using Python.

Reading the file:

```
[5]: bank = pd.read_csv('../Project1/fraudDataset/Bank_Transaction_Fraud_Detection.csv')
     bank.head()
```

[5]:

	Customer_ID	Customer_Name	Gender	Age	State	City	Bank_Branch	Account_Type	Transaction_ID	Transaction_Date	...	Merchant_C
0	d5f6ec07-d69e-4f47-b9b4-7c58ff17c19e	Osha Tella	Male	60	Kerala	Thiruvananthapuram	Thiruvananthapuram Branch	Savings	4fa3208f-9e23-42dc-b330-844829d0c12c	23-01-2025	...	Re
1	7c14ad51-781a-4db9-b7bd-67439c175262	Hredhaan Khosla	Female	51	Maharashtra	Nashik	Nashik Branch	Business	c9de0c06-2c4c-40a9-97ed-3c7b8f97c79c	11-01-2025	...	Re
2	3a73a0e5-d4da-45aa-85f3-528413900a35	Ekani Nazareth	Male	20	Bihar	Bhagalpur	Bhagalpur Branch	Savings	e41c55f9-c016-4ff3-872b-cae72467c75c	25-01-2025	...	C
3	7902f4ef-9050-4a79-857d-9c2ea3181940	Yamini Ramachandran	Female	57	Tamil Nadu	Chennai	Chennai Branch	Business	7f7ee11b-ff2c-45a3-802a-49bc47c02ecb	19-01-2025	...	Entert
4	3a4bba70-d9a9-4c5f-8b92-1735fd8c19e9	Kritika Rege	Female	43	Punjab	Amritsar	Amritsar Branch	Savings	f8e6ac6f-81a1-4985-bf12-f60967d852ef	30-01-2025	...	Entert

Checking for missing values:

```
[10]: # missing values
      bank.isnull().sum()
```

```
[10]: Customer_ID                0
      Customer_Name              0
      Gender                     0
      Age                        0
      State                      0
      City                       0
      Bank_Branch                0
      Account_Type               0
      Transaction_ID             0
      Transaction_Date           0
      Transaction_Time           0
      Transaction_Amount         0
      Merchant_ID                0
      Transaction_Type           0
      Merchant_Category          0
      Account_Balance            0
      Transaction_Device         0
      Transaction_Location       0
      Device_Type                0
      Is_Fraud                   0
      Transaction_Currency       0
      Customer_Contact           0
      Transaction_Description    0
      Customer_Email             0
      dtype: int64
```

There are no missing values in the dataset, so no cleaning was needed.

Check for duplicate values:

```
[7]: # nunique shows the number of unique values that exist in each column
     bank.nunique()
```

```
[7]: Customer_ID              200000
     Customer_Name             142699
     Gender                         2
     Age                           53
     State                         34
     City                         145
     Bank_Branch                  145
     Account_Type                   3
     Transaction_ID            200000
     Transaction_Date              31
     Transaction_Time           77856
     Transaction_Amount        197978
     Merchant_ID               200000
     Transaction_Type               5
     Merchant_Category              6
     Account_Balance           197954
     Transaction_Device             20
     Transaction_Location          148
     Device_Type                    4
     Is_Fraud                       2
     Transaction_Currency           1
     Customer_Contact            9000
     Transaction_Description      172
     Customer_Email              4779
     dtype: int64
```

For the dataset shown above, there are unique values in all of the columns; Transaction_ID, Customer_ID, and Merchant_Category have 200,000 unique values each.

Describe:

```
[9]: # describe dataset
     bank.describe()
```

[9]:

	Age	Transaction_Amount	Account_Balance	Is_Fraud
count	200000.000000	200000.000000	200000.000000	200000.000000
mean	44.015110	49538.015554	52437.988784	0.050440
std	15.288774	28551.874004	27399.507128	0.218852
min	18.000000	10.290000	5000.820000	0.000000
25%	31.000000	24851.345000	28742.395000	0.000000
50%	44.000000	49502.440000	52372.555000	0.000000
75%	57.000000	74314.625000	76147.670000	0.000000
max	70.000000	98999.980000	99999.950000	1.000000

Converting Time:

```
[17]: # convert dates
      bank['Transaction_Date'] = pd.to_datetime(bank['Transaction_Date'], format='%d-%m-%Y')
      bank['Transaction_Time'] = pd.to_datetime(bank['Transaction_Time'], format='%H:%M:%S')
```

Dataset by Gender:

```
Gender
Male      50.226
Female    49.774
Name: proportion, dtype: float64
```

Dataset by Age:

```
age_group
0-19     3.754
20s     18.792
30s     18.974
40s     18.846
50s     18.818
60+     20.816
Name: proportion, dtype: float64
```

The dataset for age groups aside from the 0-19 group, are evenly distributed.

Dataset by Date:

```
month
1    100.0
Name: proportion, dtype: float64
```

There are only transaction dates in January.

Dataset by Account Type:

```
Account_Type
Checking    33.4620
Savings     33.2965
Business    33.2415
Name: proportion, dtype: float64
```

Dataset by Merchant Category:

```
Merchant_Category
Restaurant       16.7625
Entertainment    16.7105
Electronics      16.7045
Clothing         16.6700
Groceries        16.5935
Health           16.5590
Name: proportion, dtype: float64
```

Dataset by Device Type:

```
Device_Type
POS        25.0555
ATM        25.0275
Mobile     24.9810
Desktop    24.9360
Name: proportion, dtype: float64
```

Dataset by Hour:

```
Hour
0     4.1995
1     4.1330
2     4.1560
3     4.1645
4     4.1595
5     4.1870
6     4.1605
7     4.2525
8     4.1795
9     4.0780
10    4.2000
11    4.1870
12    4.1065
13    4.1865
14    4.2260
15    4.2120
16    4.1865
17    4.1510
18    4.1475
19    4.1120
20    4.1435
21    4.1465
22    4.1970
23    4.1280
Name: proportion, dtype: float64
```

The dataset is pretty evenly split when it comes to gender, age, account type, merchant category, device type, and hour of the day of the transaction.

Amount of Fraud in Dataset:

```
Is_Fraud
0    94.956
1     5.044
Name: proportion, dtype: float64
```

```
Is_Fraud
0    189912
1     10088
dtype: int64
```

About 5% of the dataset is fraud (10,088 occurrences), meaning the dataset is unbalanced.

Account Balance Range:

```
count    200000.000000
mean      52437.988784
std       27399.507128
min        5000.820000
25%       28742.395000
50%       52372.555000
75%       76147.670000
max       99999.950000
Name: Account_Balance, dtype: float64
```

Transaction Amount Range:

```
count    200000.000000
mean      49538.015554
std       28551.874004
min          10.290000
25%       24851.345000
50%       49502.440000
75%       74314.625000
max       98999.980000
Name: Transaction_Amount, dtype: float64
```

Machine Learning Algorithm Description

Model Used

We built a simple logistic regression model using three features from the dataset:

- Transaction Amount
- Age
- Hour of Day

These features were scaled and used to predict whether a transaction was fraudulent (1) or not (0).

What the Model Can Answer

This model attempts to answer the question:

"Can basic transaction information predict whether a transaction is fraudulent?"

Results

The model achieved 95% accuracy, but this number is misleading. Fraud makes up only about 5% of the dataset, so a model can reach high accuracy by simply predicting "not fraud" for every case.

This is exactly what happened:

- 0 fraudulent transactions were correctly identified
- All 2,522 fraud cases were missed
- The confusion matrix shows predictions only in the "not fraud" category

```
Logistic Regression Results:                precision    recall  f1-score   support

           0       0.95      1.00      0.97     47478
           1       0.00      0.00      0.00      2522

    accuracy                           0.95     50000
   macro avg       0.47      0.50      0.49     50000
weighted avg       0.90      0.95      0.92     50000


 Confusion Matrix: [[47478     0]
 [ 2522     0]]
```

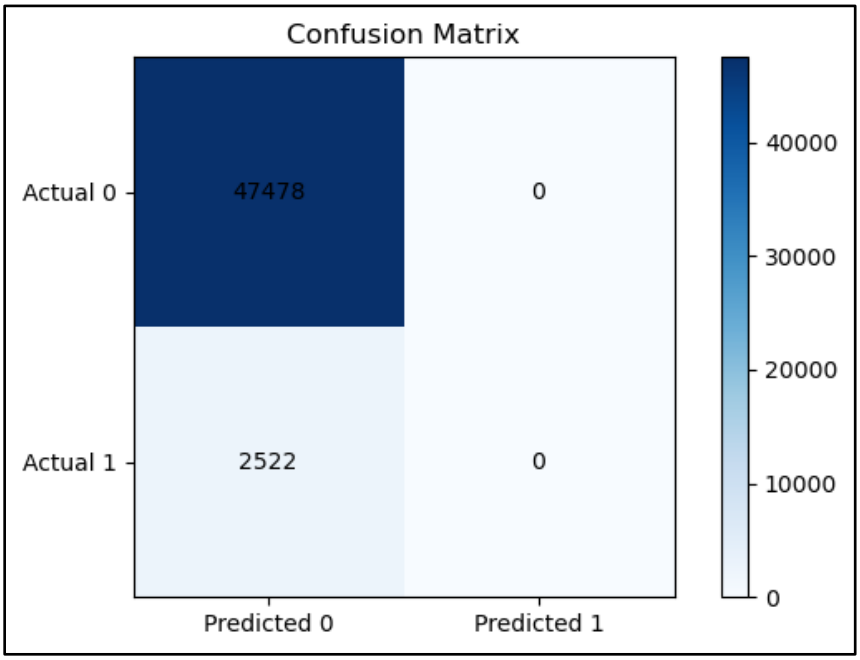
Interpretation

This model demonstrates that simple linear models are not effective for fraud detection, especially when the dataset is highly imbalanced. Logistic regression could not learn meaningful fraud patterns from the limited features provided.

Even though the model performed poorly, it highlights an important insight about the dataset. Fraud detection requires more complex features, better class balancing, or more advanced models. Basic transaction information alone is not enough to identify fraud. This aligns with real-world fraud detection challenges, where models often rely on dozens of behavioral, temporal, and account-level features.

This answers our original question:

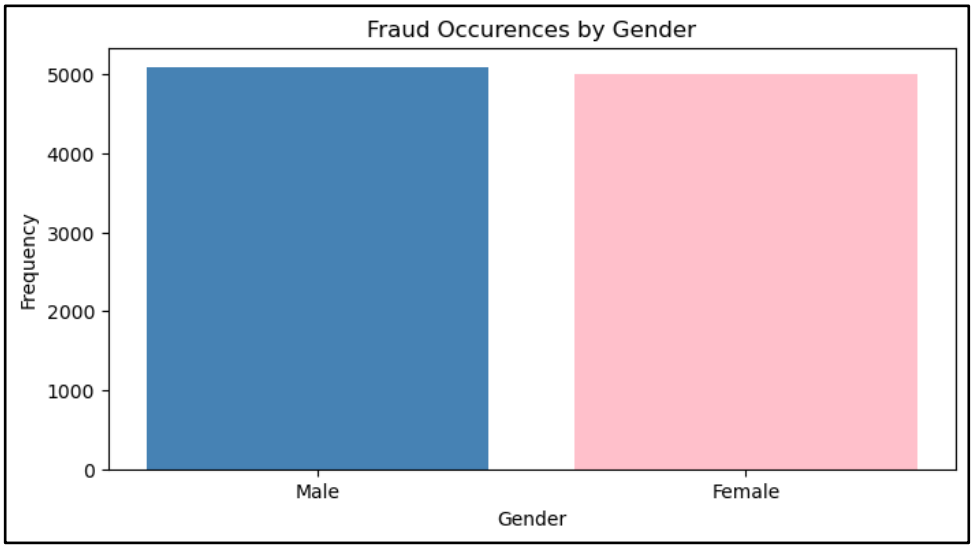
How accurate is the fraud detection?

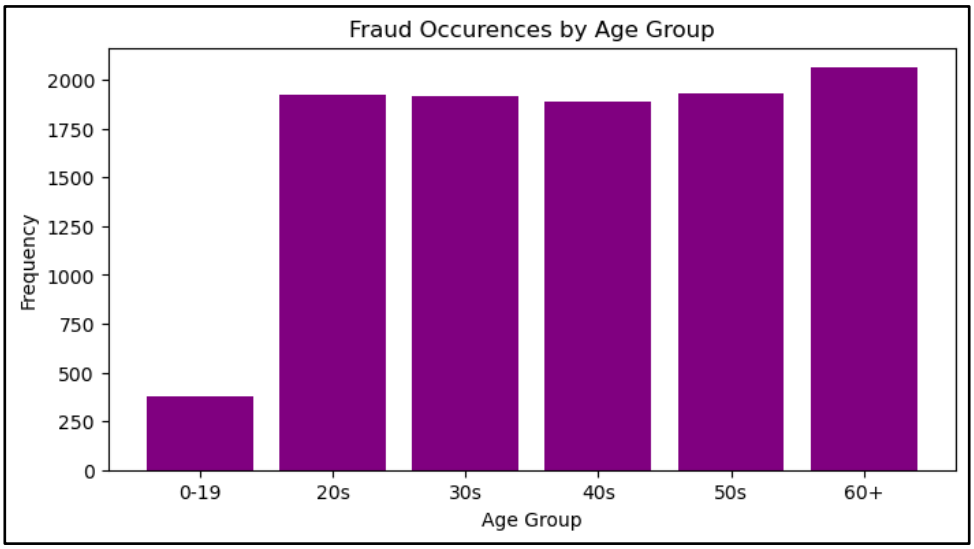


Data Visualization

Who has more fraud occurrences (gender and age)?

The following graph shows the fraud occurrences by gender. As we can see, the male population in this dataset shows to have the highest fraud occurrence in comparison to the female population. Although the difference is not as significant to show an impactful trend result, it still shows a slight impact on the overall results.

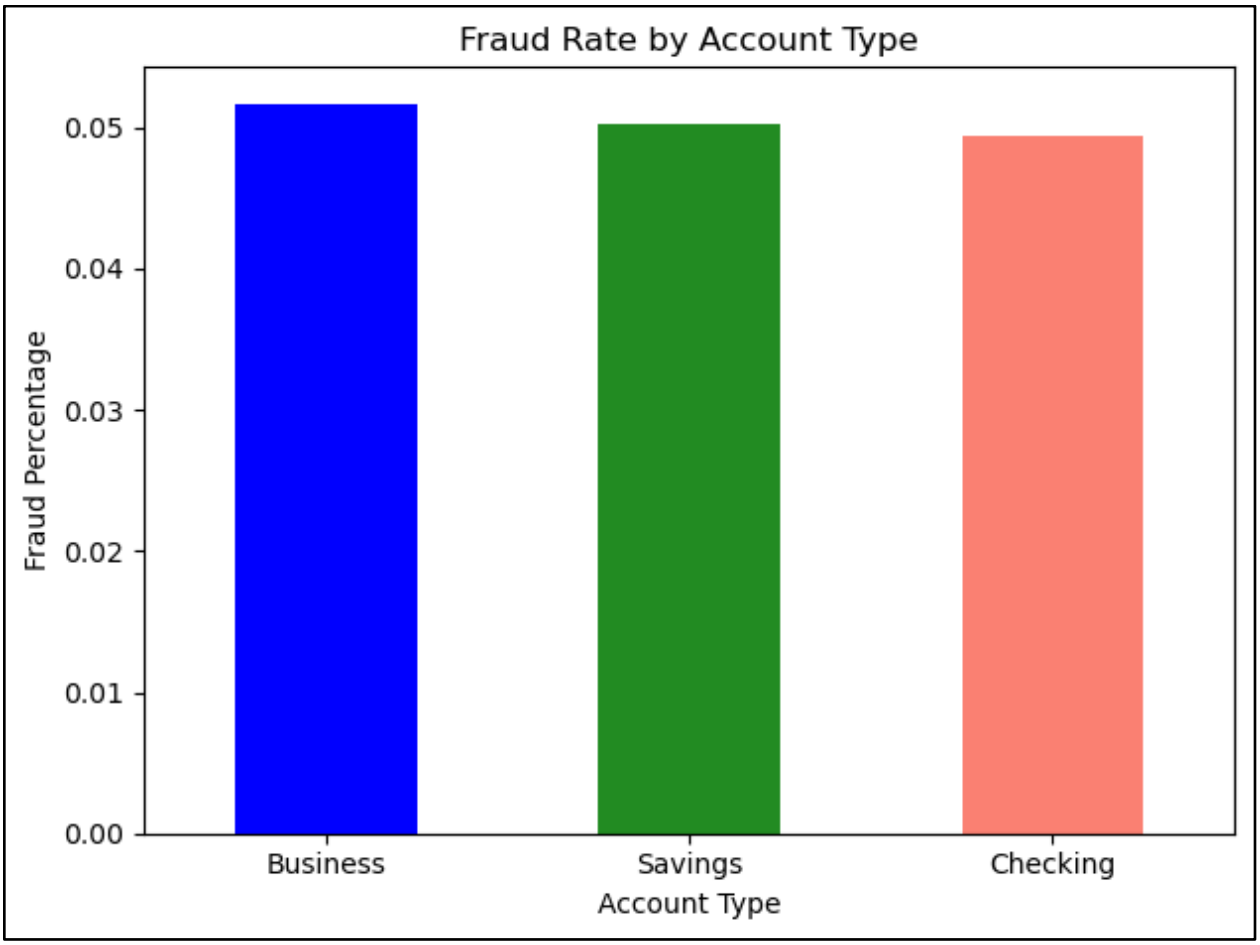




The graph above shows the fraud occurrences by age group. As shown above, the age group 60 and above show a slightly higher impact in the group totals. The age groups for 20s, 30s, 40s, and 50s also show a close result in comparison to the 60s age group. The 0-19 age range is relatively low in comparison to the other age groups but this could be due to the ages individuals are able to create and access a bank account to commit fraud.

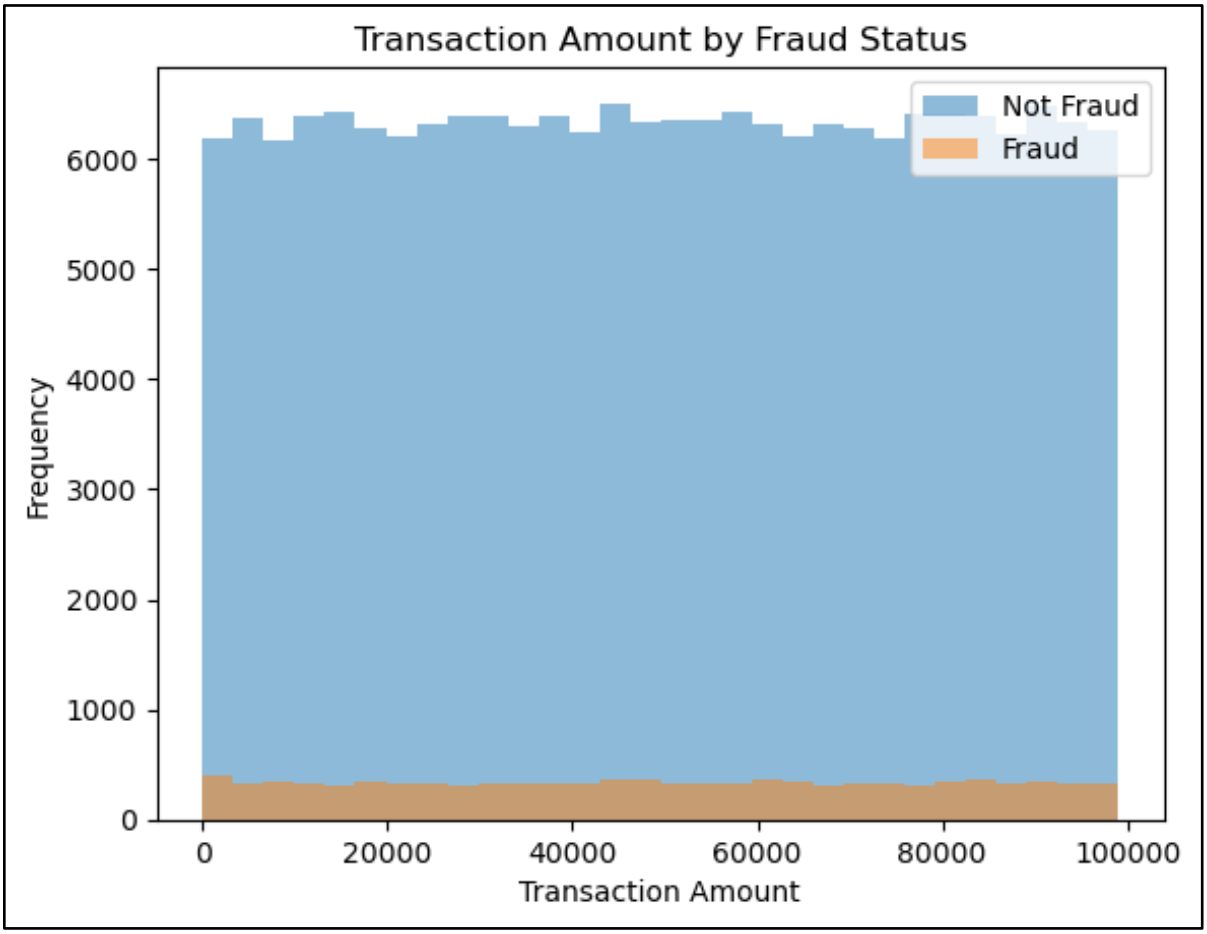
What account type has the highest report of fraudulent activities?

The graph below is showing the fraud rate by account type. We are looking at the three account types: business, savings, and checking accounts. There is not a significant difference between the three account types, but the business account type has a slightly higher difference compared to the other two account types.



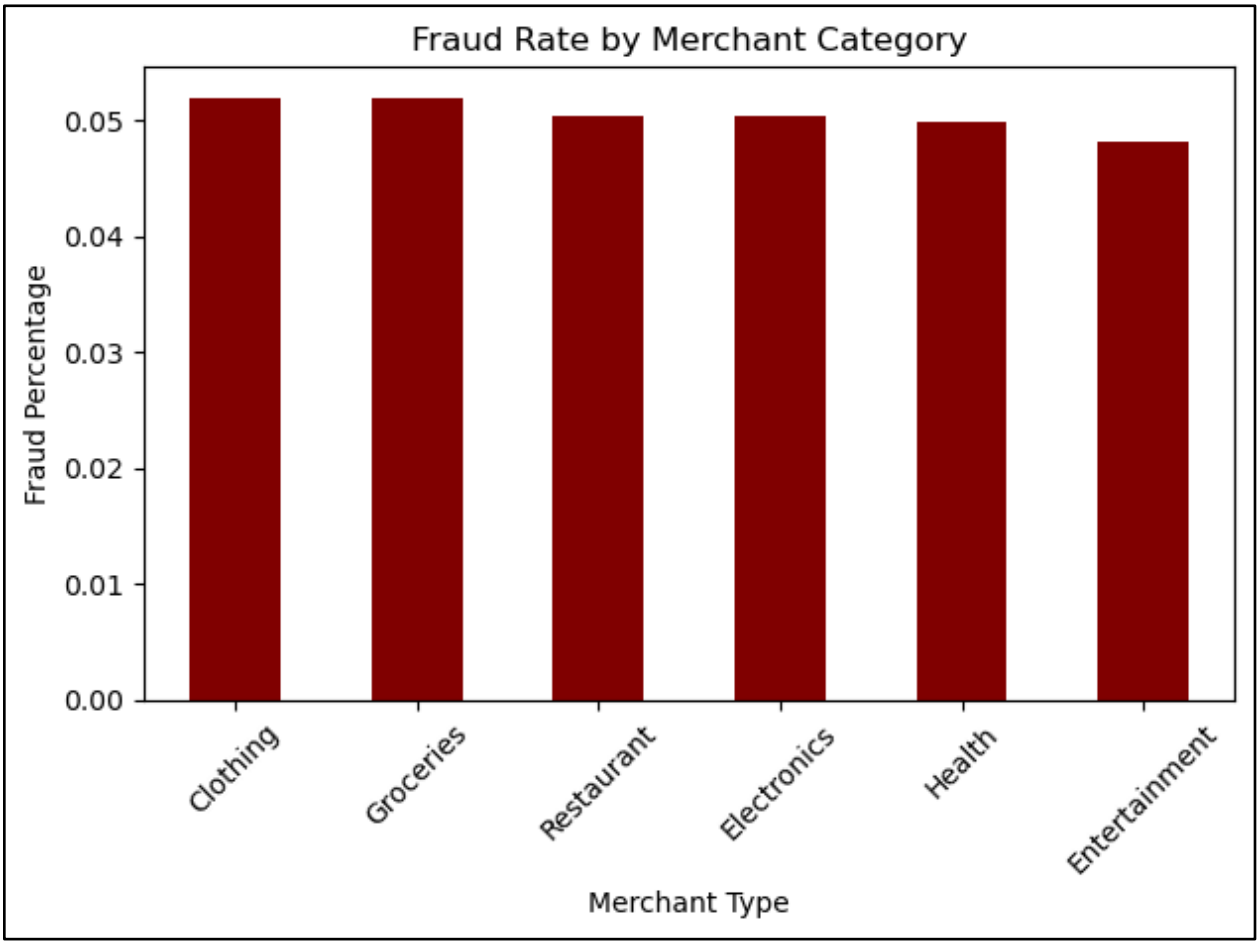
Does transaction amount have an impact on fraud?

Transaction amount does not seem to impact if fraud occurred as it is spread out evenly across the graph.



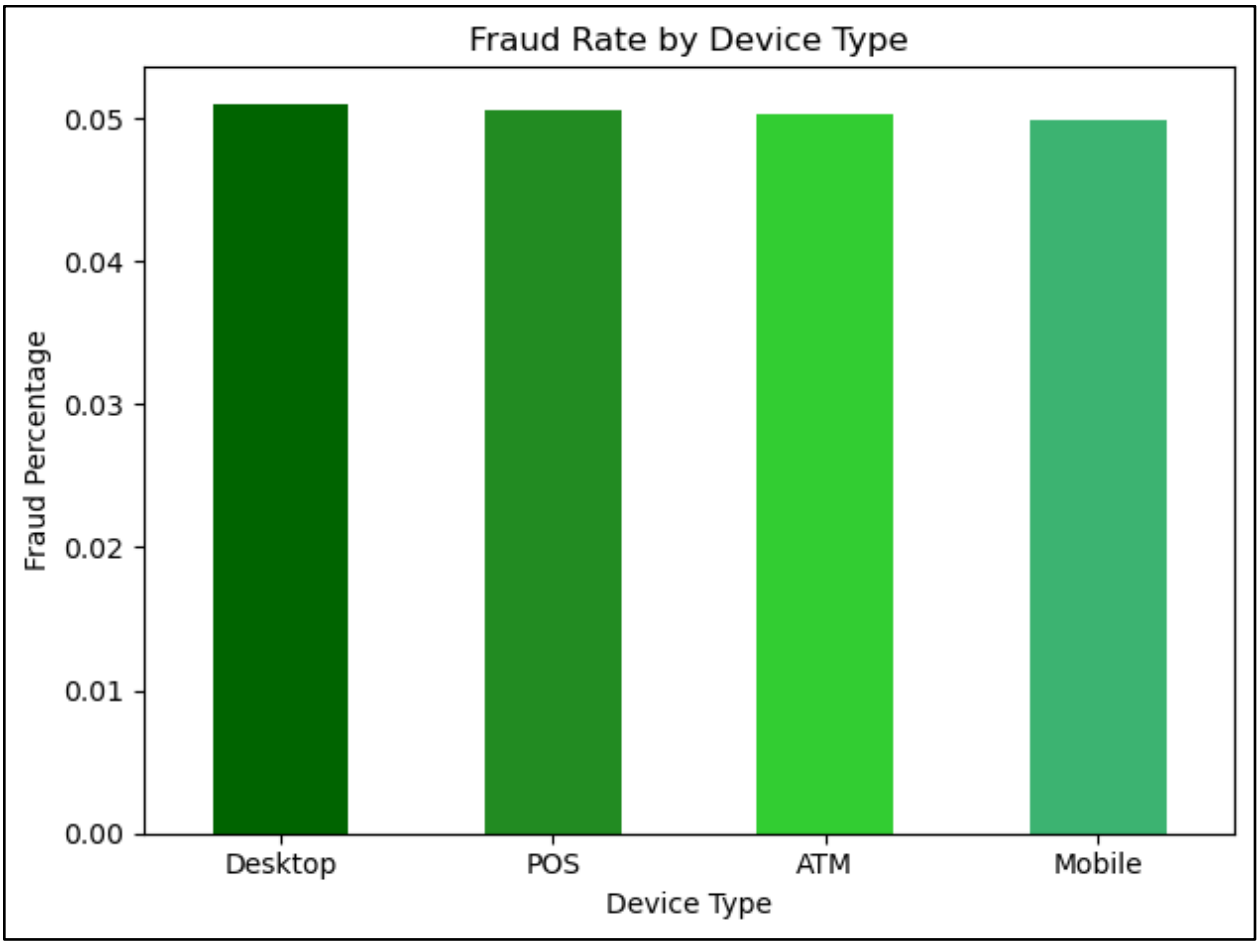
***What type of merchants have the highest report of fraud*?**

The following graph shows the fraud rate by merchant category. In the x-axis we have the merchant type and in the y-axis we have the fraud percentage. As we look at the graph below, it shows the clothing and groceries merchant type show a slight increase in the fraud percentage.



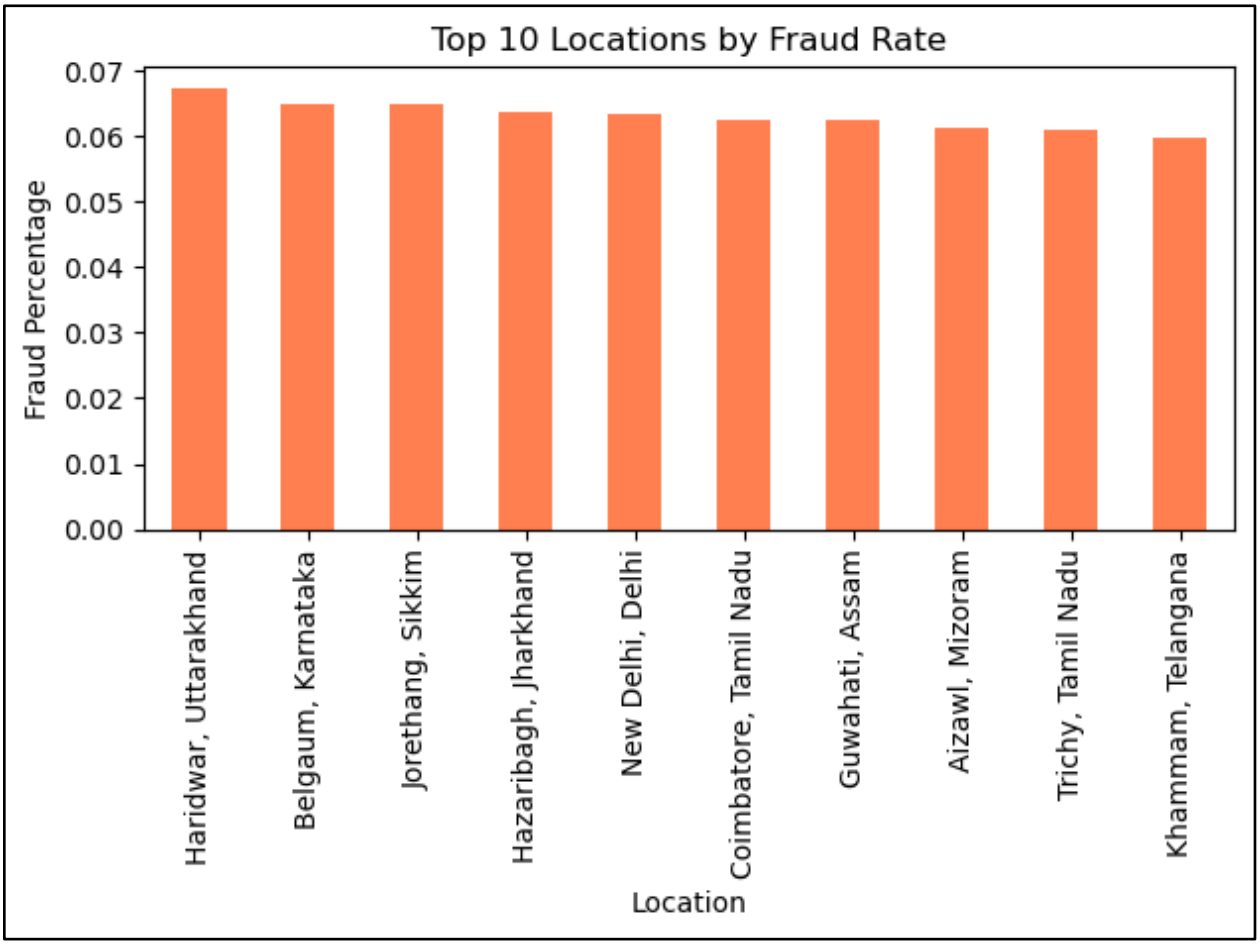
What kind of devices are being affected the most?

The graph below shows the fraud rate by device type. The x-axis is the device type and the y-axis is the fraud percentage. The four device types shown on the graph are desktop, POS, ATM, and mobile. As shown on the graph, we have a slight increase in desktop and POS device type in comparison to the other two.



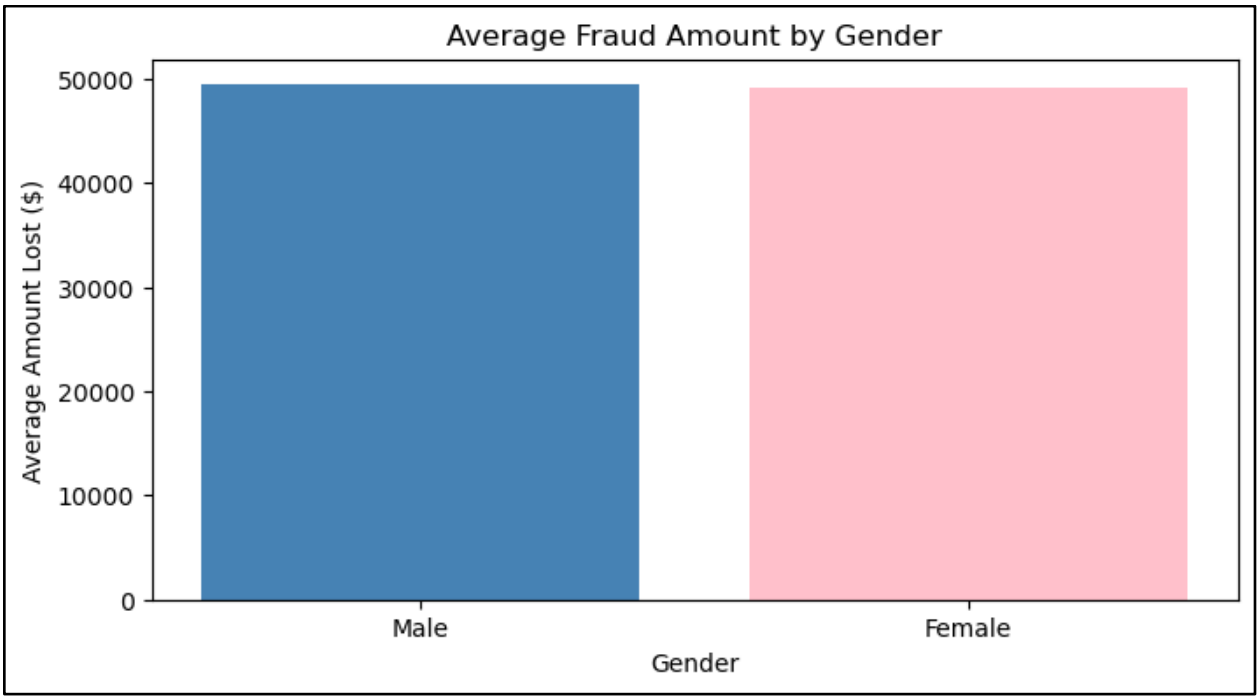
What transaction location has a higher percentage of fraud?

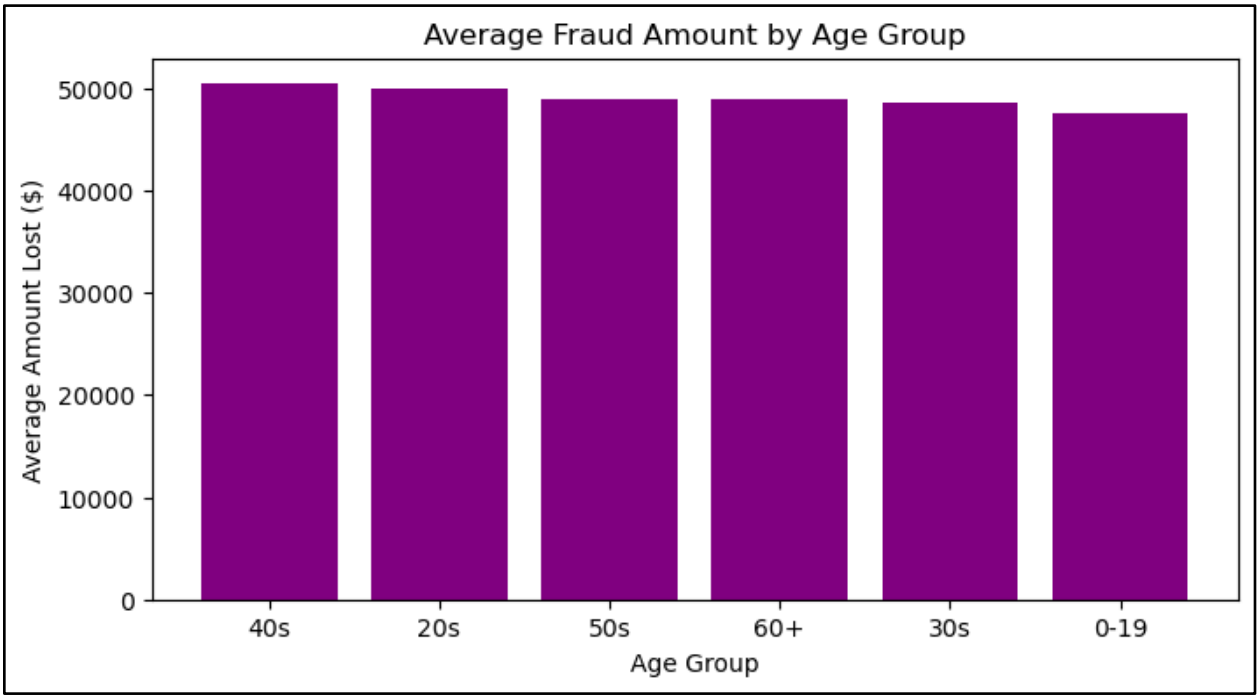
The graph below shows the top 10 locations by fraud rates. The x-axis shows the location and the y-axis shows the fraud percentage. Haridwar, Uttarakhand shows the highest fraud percentage in comparison to the other locations.

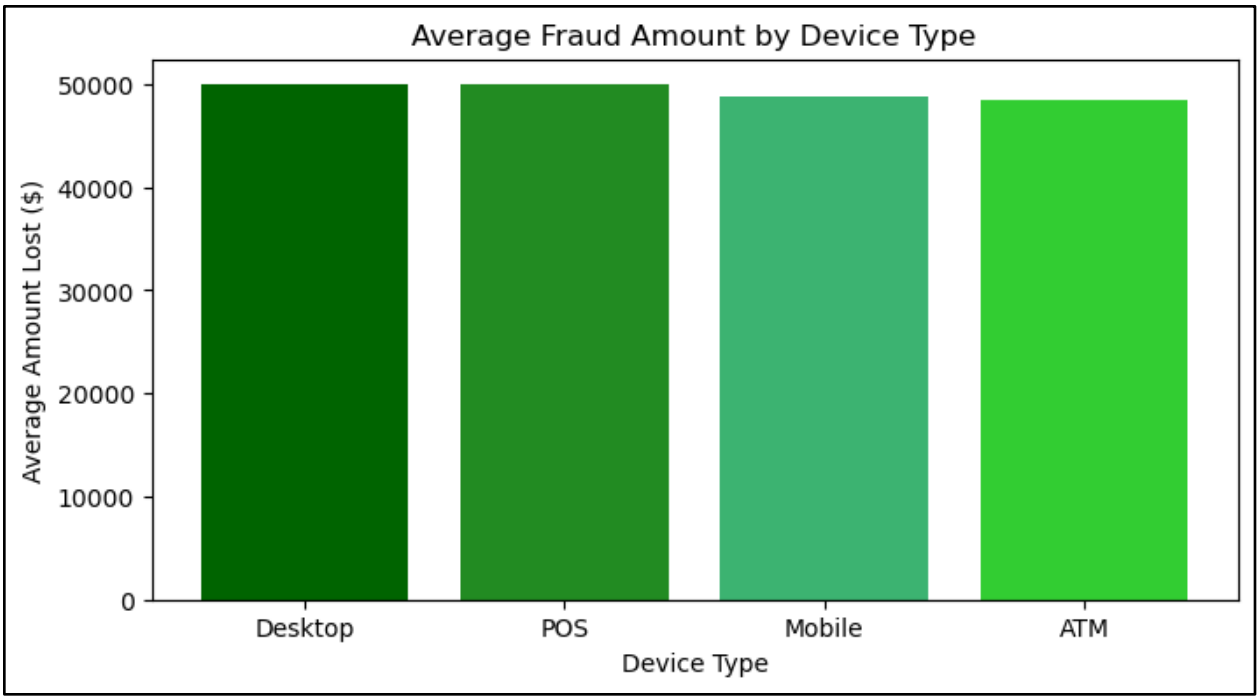


How much money is being taken by fraud based on gender, age, device type?

These graphs show the average amount of money lost in fraudulent transactions across different groups. They highlight whether certain genders, age groups, or device types tend to lose more money per fraud case. We see that there aren't huge differences among the groups, but those with the most fraud are males, people in their 40s, and desktop devices.






Average Fraud Amount by Device Type
Average Amount Lost ($)
50000
40000
30000
20000
10000
0
Desktop
POS
Mobile
ATM
Device Type

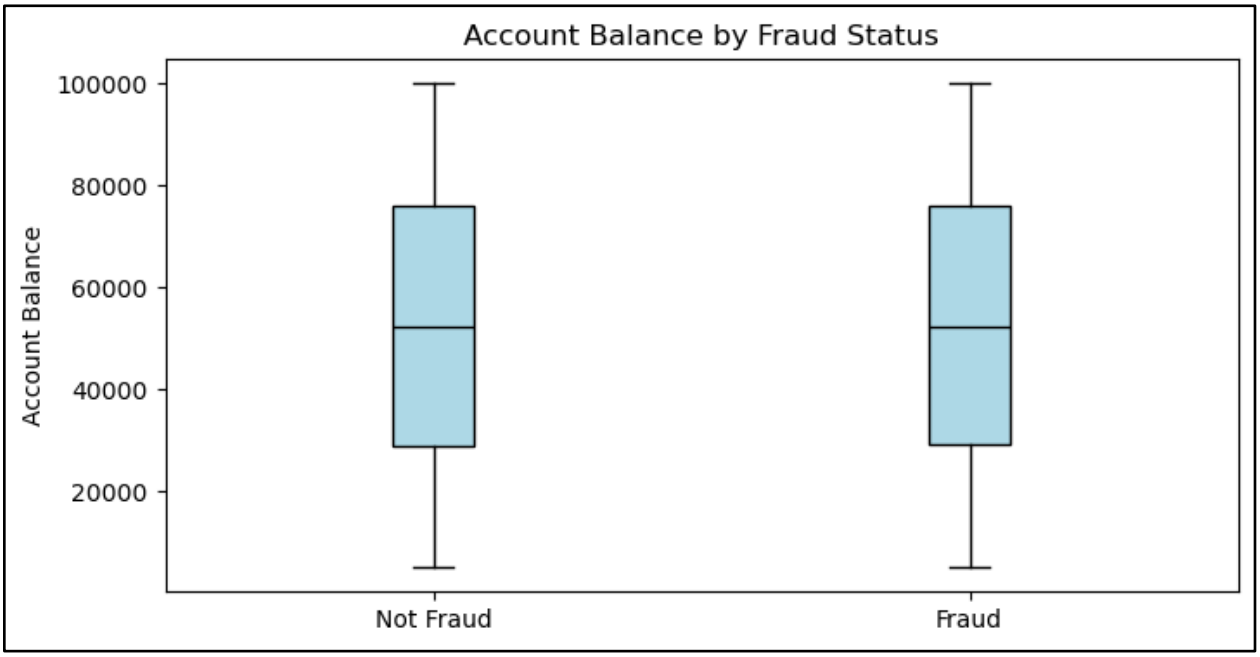
What outliers (if any) stand out and is there a specific reason why?

This summary shows the number of outliers detected in each numeric column. Most features have no outliers, but the "Is_Fraud" column appears as an outlier because fraud cases are rare compared to non-fraud cases, making the class highly imbalanced.

```
Age: 0 outliers
Transaction_Amount: 0 outliers
Account_Balance: 0 outliers
Is_Fraud: 10088 outliers
Month: 0 outliers
Hour: 0 outliers
High_Amount: 0 outliers
```

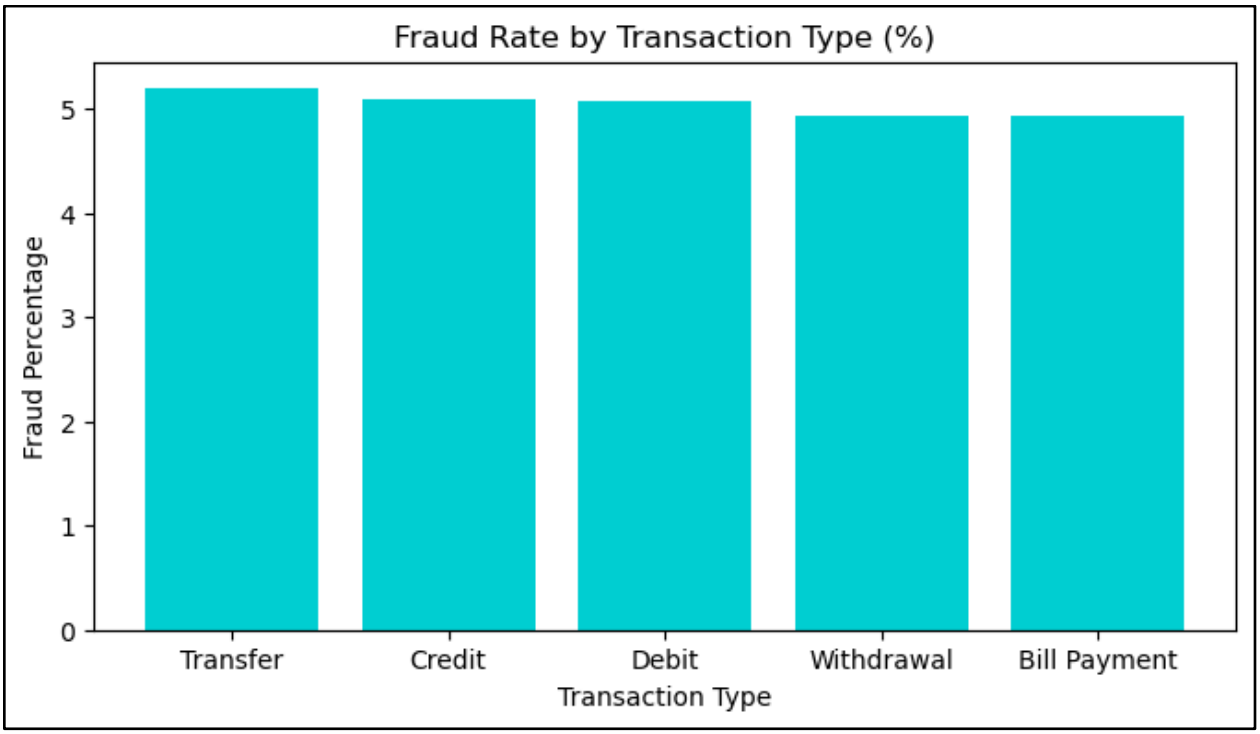
Does account balance have an impact on fraud?

This boxplot compares account balances for fraudulent and non-fraudulent transactions. The distributions look very similar, suggesting that account balance does not have a strong impact on whether fraud occurs.



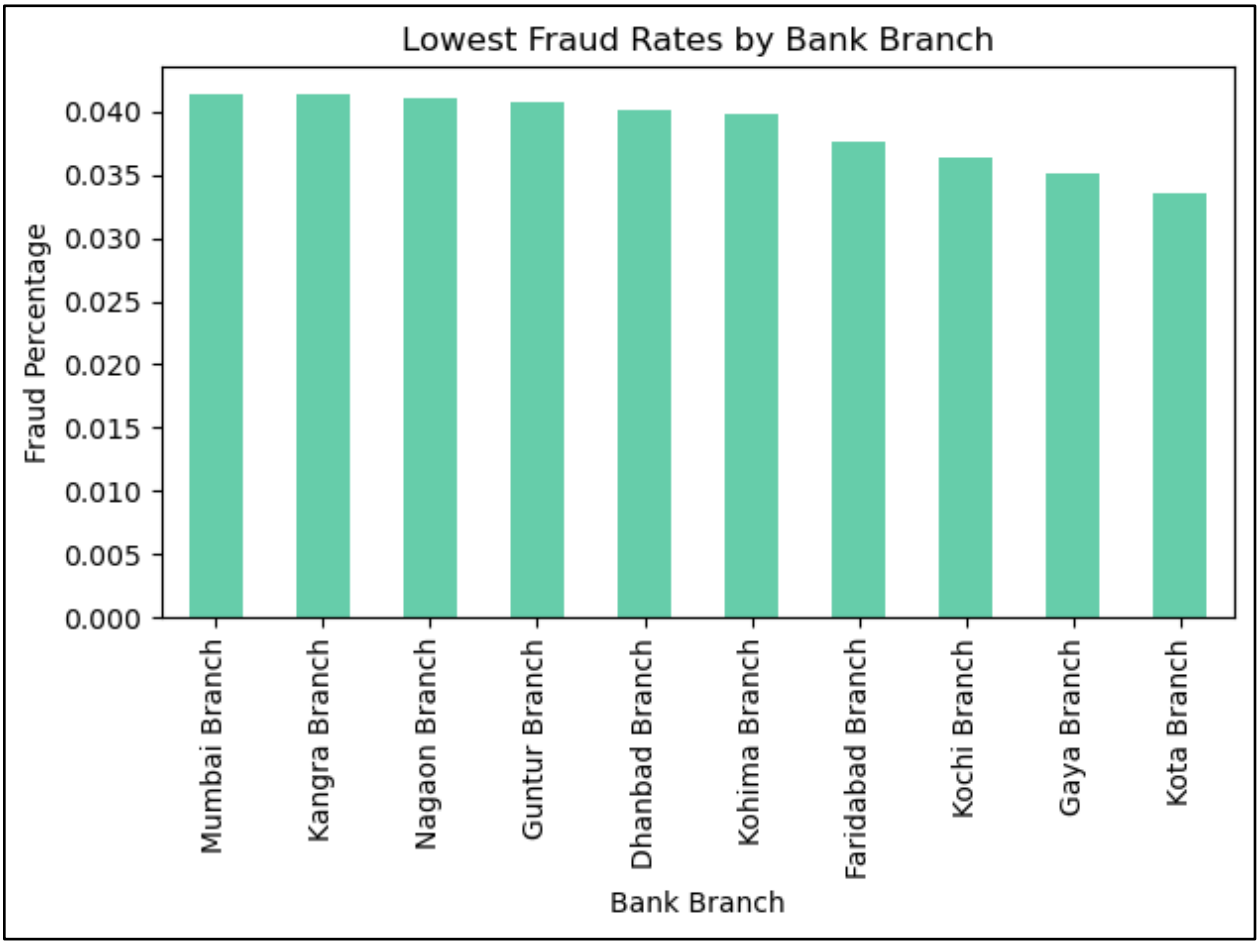
What transaction type is affected the most?

This graph shows the fraud rate for each transaction type. It highlights which types of transactions (such as transfers, withdrawals, or bill payments) experience fraud more often relative to their total volume. In this case, we see that Transfers have the highest fraud percentage (while it is close to its counterparts).



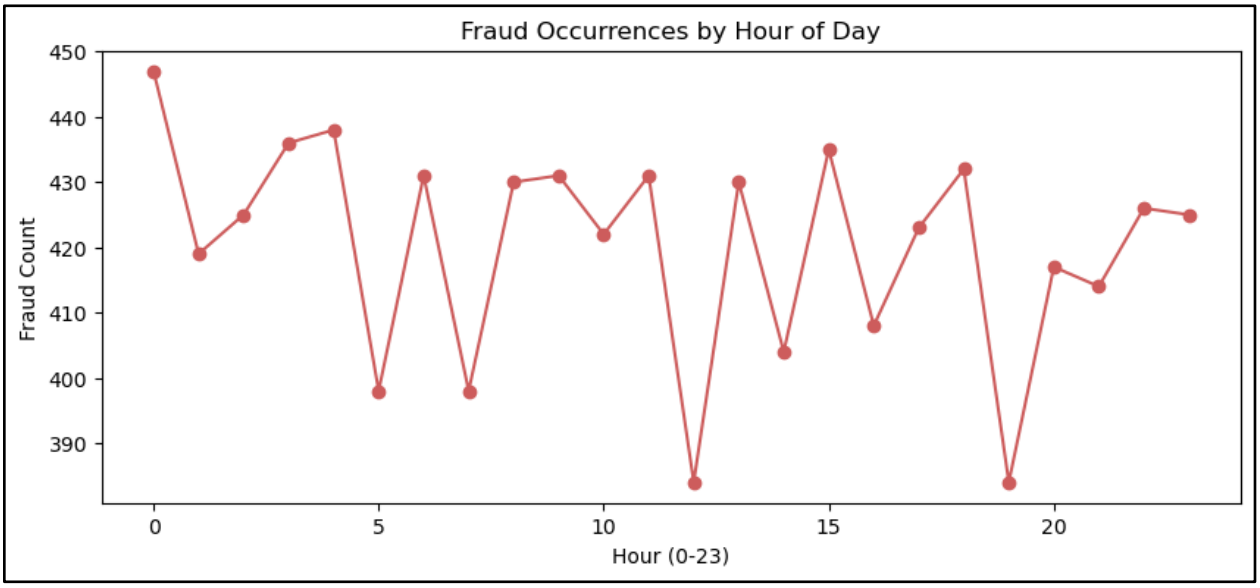
Which branches have the least amount of fraud?

The graph below shows the lowest fraud rates by bank branch. In the graph, the x-axis is the bank branch and the y-axis is the fraud percentage. The Kota Branch has one of the lowest fraud rates in comparison to the other branches in the dataset.



Is there a specific time of day that fraud occurs more often?

This graph shows how many fraudulent transactions occur at each hour of the day. The counts are pretty consistent across hours, suggesting that fraud does not cluster strongly at a specific time of day.



Appendix with Code

Imports:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# imports for machine learning model
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
```

Reading the files:

```
bank = pd.read_csv('../Project1/fraudDataset/Bank_Transaction_Fraud_Detection.csv')
bank.head()
```

Check missing values:

```
# missing values
bank.isnull().sum()
```

Duplicate values:

```
# nunique shows the number of unique values that exist in each column
bank.nunique()
```

Describe:

```
# describe dataset
bank.describe()
```

Convert Times:

```
# convert dates
bank['Transaction_Date'] = pd.to_datetime(bank['Transaction_Date'], format='%d-%m-%Y')
bank['Transaction_Time'] = pd.to_datetime(bank['Transaction_Time'], format='%H:%M:%S')
```

Dataset by Gender:

```
# even distribution between genders
bank['Gender'].value_counts(normalize = True) * 100
```

Dataset by Age:

```
# age groups
bins = [0, 19, 29, 39, 49, 59, 100]
labels = ['0-19','20s','30s','40s','50s','60+']

# create a new column with age groups
bank['age_group'] = pd.cut(bank['Age'], bins=bins, labels=labels)
bank['age_group'].value_counts(normalize = True).sort_index() * 100
```

Dataset by Date:

```
# extract month and hour
bank['Month'] = bank['Transaction_Date'].dt.month
bank['Hour'] = bank['Transaction_Time'].dt.hour
```

```
# noticed that all transactions are in January
bank['Month'].value_counts(normalize = True).sort_index() * 100
```

Dataset by Account Type:

```
# even distribution
bank['Account_Type'].value_counts(normalize = True) * 100
```

Dataset by Merchant Category:

```
# even distribution
bank['Merchant_Category'].value_counts(normalize = True) * 100
```

Dataset by Device Type:

```
# even distribution
bank['Device_Type'].value_counts(normalize = True) * 100
```

Dataset by Hour:

```
# hours seems to be evenly distributed
bank['Hour'].value_counts(normalize = True).sort_index() * 100
```

Amount of Fraud in Dataset:

```
# unbalanced, majority isn't fraudulent
bank['Is_Fraud'].value_counts(normalize = True) * 100
```

```
bank.groupby('Is_Fraud').size()
```

Account Balance Range:

```
bank['Account_Balance'].describe()
```

Transaction Amount Range:

```
bank['Transaction_Amount'].describe()
```

Machine Learning Model:

```
# select features
features = ['Transaction_Amount', 'Age', 'Hour']
X = bank[features]
y = bank['Is_Fraud']

# train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size = 0.25, random_state = 42, stratify = y
)

# scale numeric features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# convert back to df to get rid of warnings
X_train_scaled = pd.DataFrame(X_train_scaled, columns = X_train.columns)
X_test_scaled = pd.DataFrame(X_test_scaled, columns = X_test.columns)

# train a simple logistic regression model
model = LogisticRegression()
model.fit(X_train_scaled, y_train)

# predictions
y_pred = model.predict(X_test_scaled)
```

```
# results
print('Logistic Regression Results:', classification_report(y_test, y_pred, zero_division=0)) # zero_division to get rid of warning
print('\n Confusion Matrix:', confusion_matrix(y_test, y_pred))
```

Analyzing how accurate the fraud detection is:

```
# how accurate is the fraud detection
# confusion matrix heatmap
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize = (6,4))
plt.imshow(cm, cmap = 'Blues')
plt.title('Confusion Matrix')
plt.colorbar()

# tick labels
plt.xticks([0,1], ['Predicted 0', 'Predicted 1'])
plt.yticks([0,1], ['Actual 0', 'Actual 1'])

# add numbers
for i in range(2):
    for j in range(2):
        plt.text(j, i, cm[i, j], ha = 'center', va = 'center', color = 'black')

plt.tight_layout()
plt.show()
```

Analyzing who it is affecting more (Gender & Age):

```
# fraud occurrences by gender
plt.figure(figsize = (8,4))
fraud_counts = bank[bank['Is_Fraud'] == 1]['Gender'].value_counts().sort_values(ascending = False)
plt.bar(fraud_counts.index.astype(str), fraud_counts.values, color = ['steelblue', 'pink'])

plt.title('Fraud Occurences by Gender')
plt.ylabel('Frequency')
plt.xlabel('Gender')
plt.show()
```

```
# fraud occurrences by age
plt.figure(figsize = (8,4))
age_counts_fraud = bank[bank['Is_Fraud'] == 1]['age_group'].value_counts().sort_index()
plt.bar(age_counts_fraud.index, age_counts_fraud.values, color = 'purple')

plt.title('Fraud Occurences by Age Group')
plt.ylabel('Frequency')
plt.xlabel('Age Group')
plt.show()
```

Analyzing what account type has the highest report of fraud:

```
# fraud rate by account type
account_fraud = bank.groupby('Account_Type')['Is_Fraud'].mean().sort_values(ascending = False)

account_fraud.plot(kind='bar', color = ['blue', 'forestgreen', 'salmon'])
plt.title('Fraud Rate by Account Type')
plt.ylabel('Fraud Percentage')
plt.xlabel('Account Type')
plt.xticks(rotation = 0)
plt.tight_layout()
plt.show()
```

Analyzing if transaction amount has an impact on fraud:

```
# transaction amount by fraud status
plt.hist(bank[bank['Is_Fraud'] == 0]['Transaction_Amount'], bins=30, alpha=0.5, label='Not Fraud')
plt.hist(bank[bank['Is_Fraud'] == 1]['Transaction_Amount'], bins=30, alpha=0.5, label='Fraud')

plt.xlabel('Transaction Amount')
plt.ylabel('Frequency')
plt.title('Transaction Amount by Fraud Status')
plt.legend()
plt.show()
```

Analyzing what type of merchants have the highest report of fraud:

```
# fraud rate by merchant category
merchant_fraud = bank.groupby('Merchant_Category')['Is_Fraud'].mean().sort_values(ascending = False)

merchant_fraud.plot(kind='bar', color = 'maroon')
plt.title('Fraud Rate by Merchant Category')
plt.ylabel('Fraud Percentage')
plt.xlabel('Merchant Type')
plt.xticks(rotation = 45)
plt.tight_layout()
plt.show()
```

What kind of devices are being affected the most:

```
# fraud rate by device type
device_fraud = bank.groupby('Device_Type')['Is_Fraud'].mean().sort_values(ascending = False)

device_fraud.plot(kind = 'bar', color = ['darkgreen', 'forestgreen', 'limegreen', 'mediumseagreen'])
plt.title('Fraud Rate by Device Type')
plt.ylabel('Fraud Percentage')
plt.xlabel('Device Type')
plt.xticks(rotation = 0)
plt.tight_layout()
plt.show()
```

Analyzing what transaction location has a high percentage of fraud:

```
# top 10 locations based on fraud rate
location_fraud = bank.groupby('Transaction_Location')['Is_Fraud'].mean().sort_values(ascending = False)

top_locations = location_fraud.head(10)

top_locations.plot(kind = 'bar', color = 'coral')
plt.title('Top 10 Locations by Fraud Rate')
plt.ylabel('Fraud Percentage')
plt.xlabel('Location')
plt.tight_layout()
plt.show()
```

Analyzing how much money is being taken by fraud based on gender, age, device type:

```
# average fraud amount by gender
fraud_amount_gender = bank[bank['Is_Fraud'] == 1].groupby('Gender')['Transaction_Amount'].mean().sort_values(ascending = False)

plt.figure(figsize = (8,4))
plt.bar(fraud_amount_gender.index.astype(str), fraud_amount_gender.values, color = ['steelblue', 'pink'])

plt.title('Average Fraud Amount by Gender')
plt.xlabel('Gender')
plt.ylabel('Average Amount Lost ($)')
plt.show()
```

```
# average fraud amount by age group
fraud_amount_age = bank[bank['Is_Fraud'] == 1].groupby('age_group', observed = True)['Transaction_Amount'].mean().sort_values(ascending = False)

plt.figure(figsize = (8,4))
plt.bar(fraud_amount_age.index.astype(str), fraud_amount_age.values, color = 'purple')

plt.title('Average Fraud Amount by Age Group')
plt.xlabel('Age Group')
plt.ylabel('Average Amount Lost ($)')
plt.show()
```

```
# average fraud amount by device type
fraud_amount_device = bank[bank['Is_Fraud'] == 1].groupby('Device_Type', observed = True)['Transaction_Amount'].mean().sort_values(ascending = False)

plt.figure(figsize = (8,4))
plt.bar(fraud_amount_device.index.astype(str), fraud_amount_device.values, color = ['darkgreen', 'forestgreen', 'mediumseagreen', 'limegreen'])

plt.title('Average Fraud Amount by Device Type')
plt.xlabel('Device Type')
plt.ylabel('Average Amount Lost ($)')
plt.show()
```

Analyzing outliers and if there is a specific reason why:

```
# check outliers
numeric_cols = bank.select_dtypes(include = 'number').columns

def find_outliers(df, column):
    Q1 = bank[column].quantile(0.25)
    Q3 = bank[column].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = bank[(bank[column] < lower_bound) | (bank[column] > upper_bound)]

    return outliers

for col in numeric_cols:
    outliers = find_outliers(bank, col)
    print(f"{col}: {len(outliers)} outliers")
```

Analyzing if an account balance has an impact on fraud:

```
# does account balance have impact on fraud - boxplot of account balance by fraud status
fraud_bal = bank[bank['Is_Fraud'] == 1]['Account_Balance']
nonfraud_bal = bank[bank['Is_Fraud'] == 0]['Account_Balance']

plt.figure(figsize=(8,4))
plt.boxplot([nonfraud_bal, fraud_bal], boxprops = dict(facecolor = 'lightblue'),
            medianprops = {'color': 'black', 'linewidth': 1}, labels = ['Not Fraud', 'Fraud'], patch_artist = True)

plt.title('Account Balance by Fraud Status')
plt.ylabel('Account Balance')
plt.show()
```

Analyzing what transaction type is most affected by fraud:

```
# fraud rate by transaction type
fraud_by_type = bank.groupby('Transaction_Type')['Is_Fraud'].mean().sort_values(ascending = False) * 100

plt.figure(figsize = (8,4))
plt.bar(fraud_by_type.index.astype(str), fraud_by_type.values, color = 'darkturquoise')

plt.title('Fraud Rate by Transaction Type (%)')
plt.xlabel('Transaction Type')
plt.ylabel('Fraud Percentage')
plt.show()
```

Branches with the least fraud:

```
# bank branches with lowest fraud rates
branch_fraud = bank.groupby('Bank_Branch')['Is_Fraud'].mean().sort_values(ascending = False)
top_branches = branch_fraud.tail(10)
top_branches.plot(kind = 'bar', color = 'mediumaquamarine')
plt.title('Lowest Fraud Rates by Bank Branch')
plt.ylabel('Fraud Percentage')
plt.xlabel('Bank Branch')
plt.tight_layout()
plt.show()
```

Analyzing if there is a specific time of day that fraud occurs:

```
# fraud counts by hour
fraud_by_hour = bank[bank['Is_Fraud'] == 1]['Hour'].value_counts().sort_index()

plt.figure(figsize = (10,4))
plt.plot(fraud_by_hour.index, fraud_by_hour.values, 'o-', color = 'indianred')

plt.title('Fraud Occurrences by Hour of Day')
plt.xlabel('Hour (0-23)')
plt.ylabel('Fraud Count')
plt.show()
```

Findings and Conclusions

Overall, this project provided a detailed look into the patterns and behaviors surrounding fraudulent transactions at LOL Bank Pvt. Ltd. By exploring the dataset from multiple angles, like customer demographics, transaction types, device usage, merchant categories, and time of day, we were able to identify where fraud occurs most often and which groups or activities are slightly more affected. Most of the patterns showed only small differences between categories, suggesting that fraud in this dataset is fairly evenly distributed rather than concentrated in one specific area.

We also tested a simple machine learning model to see whether basic transaction information could accurately predict fraud. While the model reached high accuracy, it failed to identify any fraudulent transactions due to the strong class imbalance. This result reinforces an important point: fraud detection is a problem that requires more advanced features, better balancing techniques, or more complex models than the one we used.

Even with these limitations, the project successfully highlights the current state of fraud within the dataset and provides a baseline understanding of where risks may occur. These insights can help guide future improvements, whether that's through new data, more targeted fraud prevention strategies, or the development of stronger predictive models.

References

Bhanusri, R., & Reddy, R. (2025). Fraud Detection in Banking Transactions Using Machine Learning. *International Journal of Advance Research and Innovation*. https://doi.org/10.56595/ijari.2025016

CTI Team. (2022, December 4). *NPV vs IRR*. Corporate Finance Institute. Retrieved February 16, 2026, from https://corporatefinanceinstitute.com/resources/valuation/npv-vs-irr/

Detthamrong, U., Chansanam, W., Boongoen, T., & Iam-On, N. (2024). Enhancing Fraud Detection in Banking using Advanced Machine Learning Techniques. *International Journal of Economics and Financial Issues*. https://doi.org/10.32479/ijefi.16613

FraudNet. (2024, September 27). *Fraud Detection Using Machine Learning vs. Rules-Based Systems*. FraudNet. https://www.fraud.net/resources/fraud-detection-using-machine-learning-vs-rules-based-systems#the-basics-rules-based-systems-vs-machine-learning

Hashemi, S., Mirtaheri, S., & Greco, S. (2023). Fraud Detection in Banking Data by Machine Learning Techniques. *IEEE Access*, 11, 3034-3043. https://doi.org/10.1109 /access.2022.3232287

Hilal, W., Gadsden, S.A., & Yawney, J. (2022, May 22). *Financial Fraud: A Review of Anomaly Detection Techniques and Recent Advances*. McMaster University. https://macsphere.mcmaster.ca/items/cc4ce1d0-00ff-4907-8ebd-185ffb5e0408

Johora, F., Hasan, R., Farabi, S., Akter, J., & Mahmud, M. (2024). AI-Powered Fraud Detection in Banking: Safeguarding Financial Transactions. *The American Journal of Management and Economics Innovations*. https://doi.org/10.37547/tajmei/ volume06issue06-02

Lulka, J. (2025, December 30). *Understanding AI Fraud Detection and Prevention in 2026*. DigitalOcean. Retrieved February 18, 2026, from https://www.digitalocean.com/resources/articles/ai-fraud-detection

Maru, S. (2024). "Bank Transaction Fraud Detection" dataset. *Kaggle*. https://www.kaggle.com/datasets/marusagar/bank-transaction fraud-detection.

Morley, L. (2026, February 5). *Big Data for Fraud Detection: A Guide for Financial Services and E-commerce.* openmetal. https://openmetal.io/resources/blog/big-data-for-fraud-detection-a-guide-for-financial-services-and-e-commerce/ .

Shaik, S. (2025). Fraud Detection in Bank Transactions Using Machine Learning: A Comparative Analysis of Classification Algorithms. *2025 International Conference on Computer, Electrical & Communication Engineering (ICCECE)*, 1-7. https://doi.org/ 10.1109/iccece61355.2025.10940437